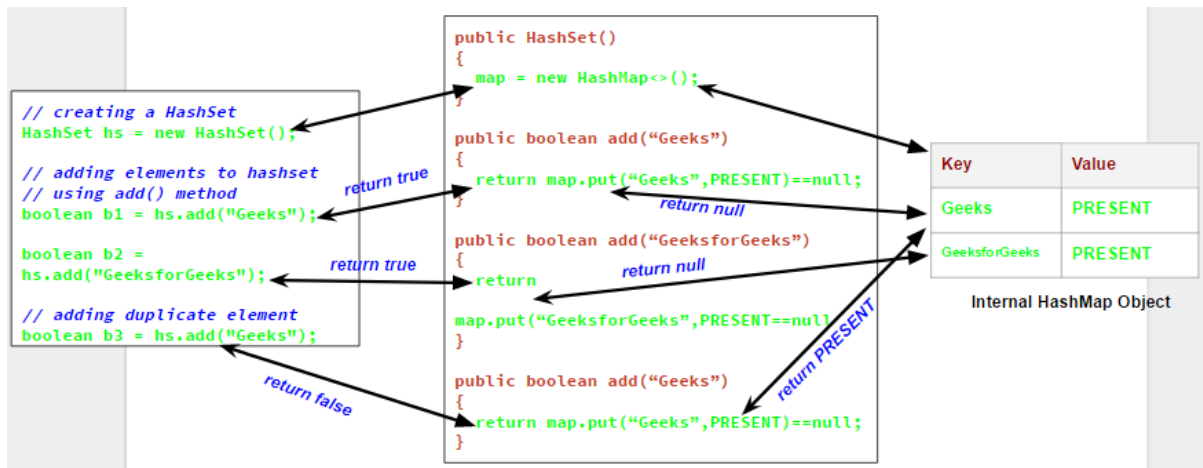


Java Collection Framework

Internal Structure of HashSet :-

- HashSet is internally implemented using a HashMap.
- Each element added to a HashSet is stored as a key in the underlying HashMap.
- The value associated with each key in the HashMap is a constant object PRESENT.
- Hashing mechanism is used to store elements — the hashCode() method determines the bucket location.



Example :-

```
import java.util.HashSet;

class Test
{
    public static void main(String args[])
    {
        // creating a HashSet
        HashSet hs = new HashSet();

        // adding elements to hashset
        // using add() method
        boolean b1 = hs.add("Geeks");
        boolean b2 = hs.add("GeeksforGeeks");

        // adding duplicate element
        boolean b3 = hs.add("Geeks");

        // printing b1, b2, b3
        System.out.println("b1 = "+b1);
        System.out.println("b2 = "+b2);
        System.out.println("b3 = "+b3);
    }
}
```

```
        // printing all elements of hashset
        System.out.println(hs);
    }
}
```

Output :-

b1 = true

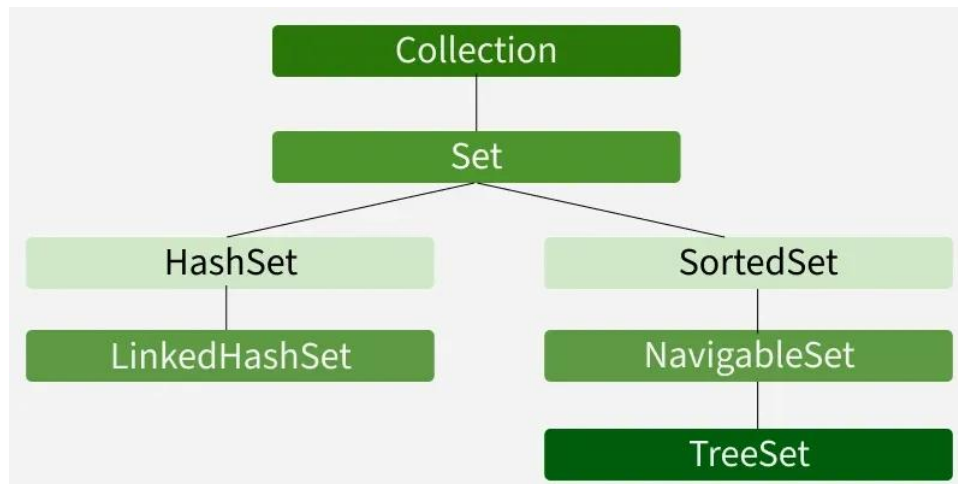
b2 = true

b3 = false

[GeeksforGeeks, Geeks]

TreeSet :-

- TreeSet in Java implements the Set interface.
- It stores elements in sorted (ascending) order.
- It is present in the java.util package.
- TreeSet does not allow duplicate elements.
- Null elements are not allowed in TreeSet.



Syntax :-

`TreeSet<Type> set = new TreeSet<Type>();`

Methods of TreeSet :-

Method	Description
<code>add(Object o)</code>	Adds element; ignores duplicates.
<code>addAll(Collection c)</code>	Adds all elements from another collection.
<code>clear()</code>	Removes all elements.
<code>clone()</code>	Returns a shallow copy of the set.
<code>contains(Object o)</code>	Checks if an element exists.
<code>first()</code>	Returns the first (lowest) element.
<code>last()</code>	Returns the last (highest) element.
<code>isEmpty()</code>	Checks if the set is empty.
<code>iterator()</code>	Returns iterator for traversal.

remove(Object o)	Removes the specified element.
size()	Returns number of elements.
headSet(Object toElement)	Returns elements less than the given element.
tailSet(Object fromElement)	Returns elements greater than or equal to the given element.
subSet(Object from, Object to)	Returns elements between given range.
descendingSet()	Returns elements in reverse order.
pollFirst()	Removes and returns first element.
pollLast()	Removes and returns last element.

Example :- using generic collection

```
package collections;

import java.util.Iterator;
import java.util.TreeSet;

public class treeset_generic_collection {

    public static void main(String[] args) throws InterruptedException {
        TreeSet<String>sweets=new TreeSet<String>();

        sweets.add("Jalebi");
        sweets.add("Ladu");
        sweets.add("Gulabjam");
        sweets.add("Pedha");
        sweets.add("Berfi");

        System.out.println("Sweets are: "+sweets);

        System.out.println("contains: "+sweets.contains("Basundi")); //contains
        System.out.println("Sweets are: "+sweets);

        System.out.println("decending_set: "+sweets.descendingSet());

        System.out.println("getfirst: "+sweets.getFirst());
        System.out.println("getlast: "+sweets.getLast());

        System.out.println("size: "+sweets.size());
        System.out.println("floor: "+sweets.floor("Gajar Halwa"));
        System.out.println("first: "+sweets.first());
        System.out.println("lower: "+sweets.lower("Rasmalai"));
        System.out.println("higher: "+sweets.higher("Bundi"));

        System.out.println("pollfirst: "+sweets.pollFirst()); //pollfirst
        System.out.println("After pollfirst sweets are: "+sweets);

        System.out.println("polllastst: "+sweets.pollLast()); //polllast
        System.out.println("After polllast sweets are: "+sweets);

        System.out.println(sweets.subSet("Gulabjam","Ladu"));

        System.out.println("isempty: "+sweets.isEmpty());

        System.out.println("tailset: "+sweets.tailSet("Gulabjam"));

        System.out.println("Iterating....Sweets.....");
        Iterator it=sweets.iterator();
        while(it.hasNext())
        {
            System.out.println(it.next());
            Thread.sleep(1000);
        }
    }
}
```

```
sweets.clear(); //clear
System.out.println("Sweets are: "+sweets);

}

}
```

Output :-

```
Sweets are: [Berfi, Gulabjam, Jalebi, Ladu, Pedha]
contains: false
Sweets are: [Berfi, Gulabjam, Jalebi, Ladu, Pedha]
decending_set: [Pedha, Ladu, Jalebi, Gulabjam, Berfi]
getfirst: Berfi
getlast: Pedha
size: 5
floor: Berfi
first: Berfi
lower: Pedha
higher: Gulabjam
polfirst: Berfi
After pollfirst sweets are: [Gulabjam, Jalebi, Ladu, Pedha]
pollastst: Pedha
After polllast sweets are: [Gulabjam, Jalebi, Ladu]
[Gulabjam, Jalebi]
isempty: false
tailset: [Gulabjam, Jalebi, Ladu]
Iterating....Sweets.....
Gulabjam
Jalebi
Ladu
Sweets are: []
```

Example :- using non-generic collection

```
package collections;

import java.util.TreeSet;

public class treeset_nongeneric_collection {

    public static void main(String[] args) {
        TreeSet ts=new TreeSet();
        ts.add(101);
        ts.add("Sagar");
        ts.add(12345678901);
        ts.add(3200.00);

        System.out.println("Employee Details are: "+ts);
    }
}
```

Output:-

Exception in thread "main" java.lang.ClassCastException

Note:

Non-generic collections are not type-safe and can cause runtime errors.

That's why generic collections are preferred in Java — they ensure type safety, readability, and compile-time error checking.

Difference Between HashSet, LinkedHashSet & TreeSet :-

Feature	HashSet	LinkedHashSet	TreeSet
Order	Does not maintain any order.	Maintains insertion order.	Maintains elements in sorted (ascending) order.
Implementation	Uses HashTable internally.	Uses LinkedHashMap internally.	Uses TreeMap internally.
Null Elements	Allows one null element.	Allows one null element.	Does not allow null element.
Performance	Fastest for add, remove, search ($O(1)$).	Slightly slower than HashSet ($O(1)$).	Slower ($O(\log n)$) due to sorting.
Sorting	No sorting of elements.	No sorting; preserves insertion order.	Automatically sorts elements in ascending order.
Use Case	When order doesn't matter.	When insertion order must be preserved.	When sorted data is required.